【手猫研发提效随笔】七句话搞定一年AB实验下线：CodingAgent的魔法时刻

原创 天猫技术团队 大淘宝技术 2026年7月31日 01:01 浙江 3人

本文分享了作者利用AI助手高效治理积压AB实验的实战经验，通过七步指令将原本繁琐的手动下线工作转化为自动化流程。核心方法是将复杂任务拆解为创建查询、扫描报告及版本升级等专用Skill工具，利用API批量获取实验状态并生成Markdown报告，随后以最小风险策略（将Key置空）批量固化代码，最终自动生成提测报告。文章强调了“工具化关键步骤”、“明确指令”及“人工Review”的重要性，并总结了API适配、JSON解析及版本冲突等踩坑教训，指出AI并非替代程序员，而是作为超级助手增强研发效能，帮助开发者从重复劳动中解放精力。

01 前言：实验太多也是一种烦恼

话说咱们团队过去一年搞了那么多AB实验，现在回头一看，好家伙，实验列表比我的购物清单还长！这些实验有的已经完成了使命，有的早就该下线了，但就是没人去收拾这个烂摊子。

直到又一次，因为预发环境AB实验过期（线上实验会自动续期，但是预发不会呀~~~）而苦逼的查了半天，这实验治理说什么都要搞了！！！

我看着那长长的实验列表，心里一万只草泥马奔腾而过。手动一个一个下线？那得搞到猴年马月！正当我准备写辞职信的时候，我想起了CodingAgent——那个号称"什么都能干"的AI助手。

02 七句话的魔法时刻

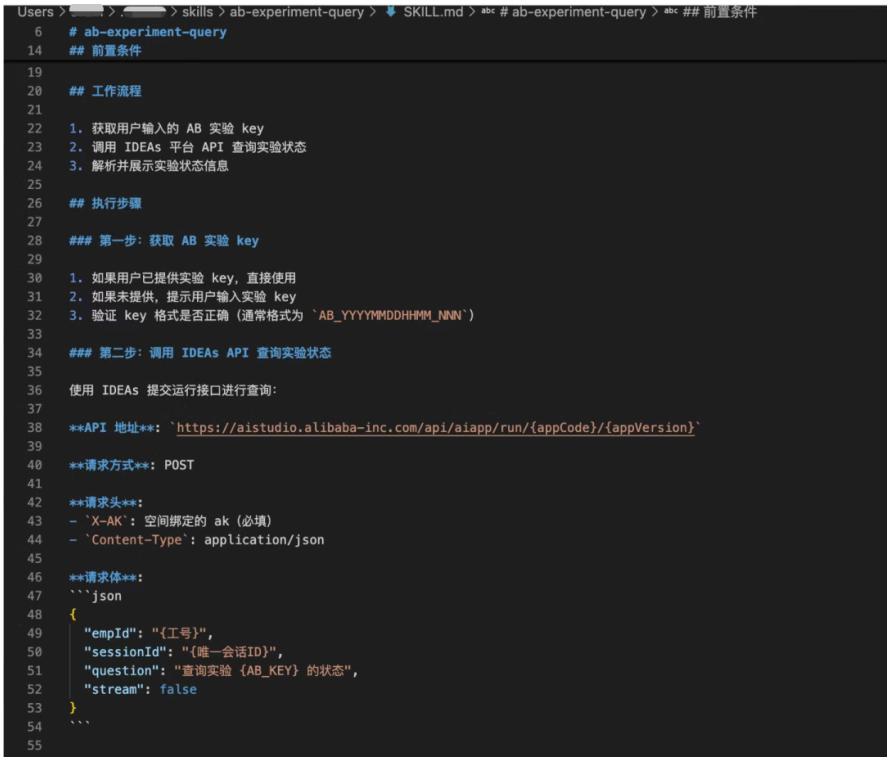
于是，我对CodingAgent说了这七句话：

第一句： "CodingAgent，帮我创建一个skill，可以根据AB实验的key查询当前AB实验状态。"



CodingAgent二话不说，直接创建了一个

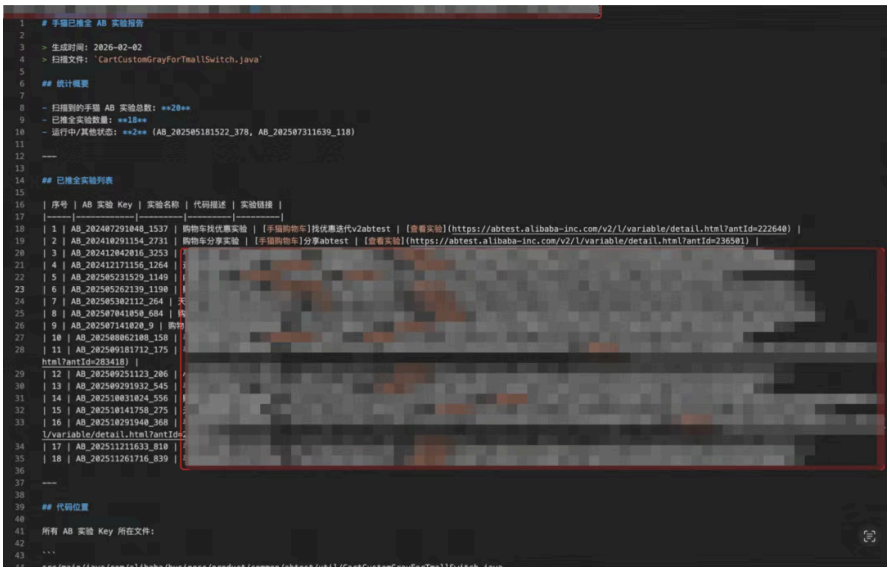
`ab-experiment-query` skill，通过IDEAs平台的API查询实验状态。我给它提供了我的AK、工号和appCode，它就能像查户口一样把实验的状态查得清清楚楚。



第二句："扫描代码统计已推全的手猫实验，生成报告。报告内容包括：ab实验key，实验描述，代码路径，实验链接。"



CodingAgent立马开始干活，先是用grep从代码里扒出了20个AB实验的key，然后批量调用API查询状态。20个实验，18个已推全，2个还在运行中。它还贴心地生成了一个markdown报告，表格排得整整齐齐，比我做的PPT还好看。



```
# 手猫已推全 AB 实验报告

> 生成时间：2026-02-02
> 扫描文件：`CartCustomGrayForTmallSwitch.java`

## 统计概要

- 扫描到的手猫 AB 实验总数：**20**
- 已推全实验数量：**18**
- 运行中/其他状态：**2** (AB_202505181522_378, AB_202507311639_118)

---

## 已推全实验列表

| 序号 | AB 实验 Key | 实验名称 | 代码描述 | 实验链接 |
|---|---|---|---|---|
| 1 | AB_202407291048_1537 | 购物车找优惠实验 | 【手猫购物车】找优惠迭代v2abtest | 【查看实验】(https://abtest.alibaba-inc.com/v2/l/variable/detail.html?antId=222640) |
| 2 | AB_202410291154_2731 | 购物车分享实验 | 【手猫购物车】分享abtest | 【查看实验】(https://abtest.alibaba-inc.com/v2/l/variable/detail.html?antId=236501) |
| 3 | AB_202412042016_3253 | [redacted] |
| 4 | AB_202412171156_1264 | [redacted] |
| 5 | AB_202505231529_1149 | [redacted] |
| 6 | AB_202505262139_1190 | [redacted] |
| 7 | AB_202505302112_264 | [redacted] |
| 8 | AB_202507041058_684 | [redacted] |
| 9 | AB_202507141020_9 | [redacted] |
| 10 | AB_202508062108_158 | [redacted] |
| 11 | AB_202509181712_175 | [redacted] |
html?antId=283418) |
| 12 | AB_202509251123_206 | [redacted] |
| 13 | AB_202509291932_545 | [redacted] |
| 14 | AB_202510031024_556 | [redacted] |
| 15 | AB_202510141758_275 | [redacted] |
| 16 | AB_202510291940_368 | [redacted] |
l/variable/detail.html?antId=
| 17 | AB_202511211633_810 | [redacted] |
| 18 | AB_202511261716_839 | [redacted] |

---

## 代码位置

所有 AB 实验 Key 所在文件：

```
```

第三句： "将这个扫描能力也创建成skill。"

于是`scan-ab-experiments` skill诞生了。以后只要输入`/scan-ab-experiments`，它就能自动扫描代码、查询状态、生成报告，全程自动化，我只需要喝茶看结果。
```

```
# scan-ab-experiments
## 工作流程

## 执行步骤

### 第一步：确定扫描范围

询问用户要扫描的范围：
- 特定文件（如 `CartCustomGrayForTmallSwitch.java`）
- 特定目录（如 `src/main/java/...`）
- 特定模式（如只扫描手猫相关：`*Tmall*.java`）

如果用户未指定，默认扫描当前项目中所有 Switch 文件。

### 第二步：提取 AB 实验 Key

使用 grep 命令提取所有 AB 实验 key：

```bash
grep -rhoE 'AB_[0-9]{12}_[0-9]+' <扫描路径> | sort -u
```

AB 实验 key 格式：`AB_YYYYMMDDHHMM_NNN`

第三步：批量查询实验状态

对每个实验 key，调用 IDEAs API 查询状态：

```bash
curl -s -X POST "https://aistudio.alibaba-inc.com/api/aiapp/run/{appCode}/latest" \
  -H "X-AK: {ak}" \
  -H "Content-Type: application/json" \
  -d '{
    "empId": "{empId}",
    "sessionId": "unique-id",
    "question": "查询实验 {AB_KEY} 的状态",
    "stream": false
  }'
```

从响应中判断实验状态：
- 包含 "已推全" → 已推全
- 包含 "100%" 且 "对照组.*0%" → 已推全
- 包含 "运行中/灰度中/进行中" → 运行中
- 其他 → 其他状态
```

第四句： "帮我将当前二方库版本升级并设置为SNAPSHOT版本。"



CodingAgent看了看pom.xml，发现当前版本是2.10.51-RELEASE-2026012902，二话不说就升级成了2.10.52-SNAPSHOT。我还让它加了分支标识，避免和其他开发者的版本冲突。



第五句： "将上述操作创建一个skill给我，方便下次快速执行。"

`upgrade-snapshot` skill闪亮登场！以后每次开发新功能，只要输入`/upgrade-snapshot`，它就能自动升级版本号、提取分支标识、生成新版本号，全程不用我操心。

Users > evain > .claude > skills > upgrade-snapshot > SKILL.md > # upgrade-snapshot > ## 示例

```
---
name: upgrade-snapshot
description: 升级二方库版本为 SNAPSHOT 版本，并添加分支标识避免冲突
---

# upgrade-snapshot

升级二方库版本为 SNAPSHOT 版本，并添加分支标识避免冲突。

## 触发方式

使用 `/upgrade-snapshot` 命令触发

## 执行步骤

1. 读取项目根目录的 `pom.xml` 文件，获取当前版本号
2. 解析当前版本号，提取主版本号（如 `2.10.51`）
3. 将 patch 版本号 +1（如 `2.10.51` -> `2.10.52`）
4. 获取当前 git 分支名，从分支名中提取功能标识：
   - 分支格式通常为 `feature/YYYYMMDD_XXXXXXXX_功能名_N`
   - 提取 `功能名` 部分作为标识（去掉日期、工单号和末尾序号）
5. 生成新版本号格式：`{主版本}.{次版本}.{patch+1}-{分支标识}-SNAPSHOT`
6. 更新 `pom.xml` 中项目自身的 `<version>` 标签（注意不要修改 parent 的版本）

## 版本号解析规则

- 原版本格式：`X.Y.Z-RELEASE-YYYYMMDDNN` 或 `X.Y.Z-SNAPSHOT` 或 `X.Y.Z-xxx-SNAPSHOT`
- 提取 `X.Y.Z` 作为基础版本号
- patch 版本 Z 加 1

## 分支标识提取规则

- 输入：`feature/20260202_28248387_tmallRemoveAb2026_1`
- 按 `_` 分割后，跳过日期（第1段）和工单号（第2段），取功能名部分（第3段起，去掉末尾的纯数字序号）
- 输出：`tmallRemoveAb2026`

## 示例

- 原版本：`2.10.51-RELEASE-2026012902`
- 分支：`feature/20260202_28248387_tmallRemoveAb2026_1`
- 新版本：`2.10.52-tmallRemoveAb2026-SNAPSHOT`
```

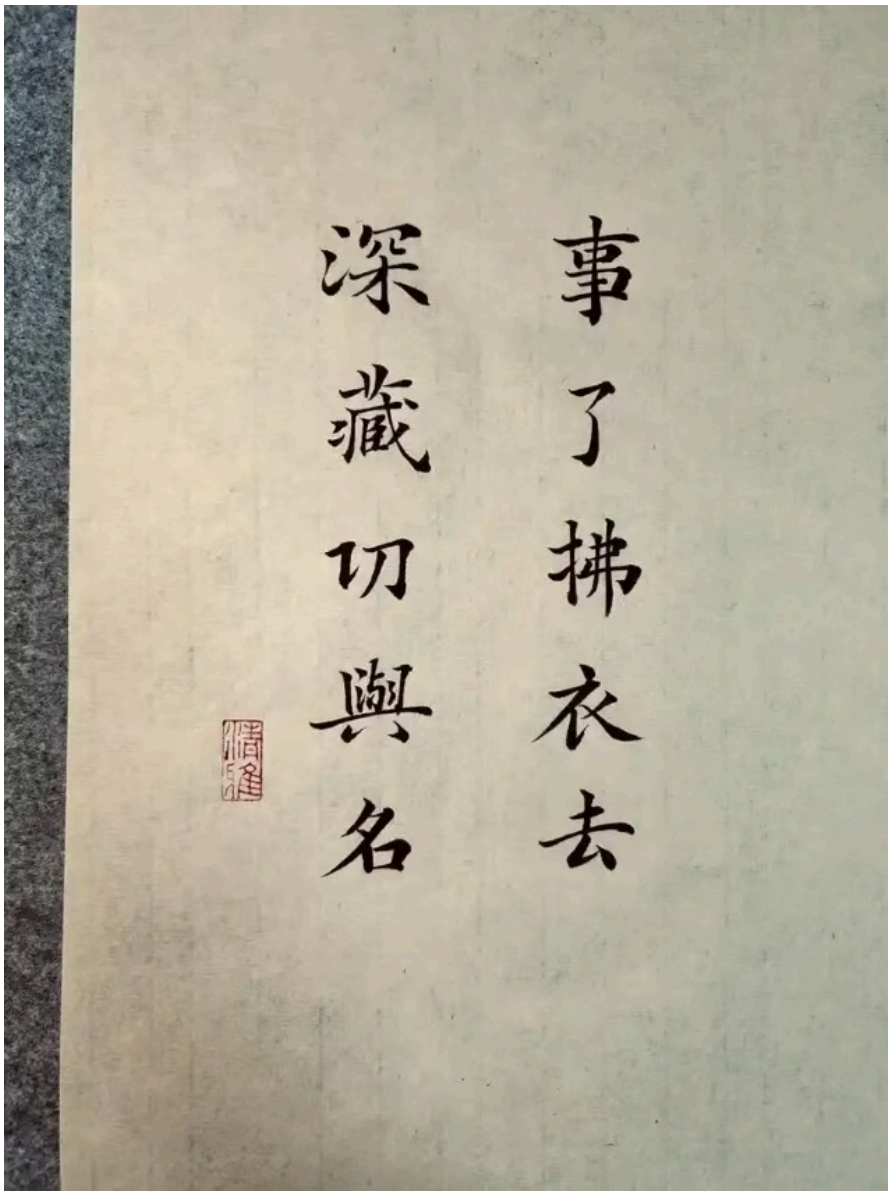
第六句： "把报告文件中已推全的实验逻辑，固化到代码中。并将固化逻辑生成md报告给我。"



CodingAgent进入计划模式，分析了18个已推全实验的代码结构，然后问我："大哥队长，这些实验怎么固化？设置为空值还是删除判断逻辑？"我选择了设置为空值，这样代码改动最小，风险最低。CodingAgent就开始批量修改代码，每个实验key都改成空字符串，还加了注释说明原值。



第七句： "生成最终的提测报告，准备上线。"



CodingAgent汇总了所有操作，生成了一份完整的提测报告，包括修改的文件、固化的实验列表、风险评估、测试建议。我看了看报告，嗯，挺专业的，可以直接发给测试同学了。

天猫 AB 实验固化提测报告

> 提测时间：2026-02-02
> 分支：`feature/20260202_28248387_tmallRemoveAb2026_1`
> 模块：`china-carts-products-common`

一、变更背景

根据 AB 实验平台数据，18 个已推全的 AB 实验需要进行代码固化，移除 AB 实验判断逻辑，保留实验组（推全）结果。

二、变更范围

2.1 涉及文件

文件	变更类型
`CartCustomGrayForTmallSwitch.java`	开关配置变更
`CartCustomGrayForTmallUtil.java`	AB 逻辑清理
`MixCartUtilForTmall.java`	AB 逻辑清理
`TmallConsumeCouponPriceUtil.java`	AB 逻辑清理

2.2 固化实验列表（共 18 个）

序号	AB 实验 Key	实验名称	涉及开关/变量	固化方式
1	AB_202407291048_1537	购物车找优惠实验	TMALL_FIND_PROMOTION_V2_AB_COMPONENT_ID	注释代码，返回 true
2	AB_202410291154_2731	购物车分享实验	TMALL_GET_SHARE_AB_COMPONENT_ID	注释代码，返回 true
3	AB_202412042016_3253	[redacted]		
4	AB_202412171156_1264			
5	AB_202505231529_1149			
6	AB_202505262139_1190			
7	AB_202505302112_264			
8	AB_202507041050_684			
9	AB_202507141020_9	服		
10	AB_202508062108_158			
11	AB_202509181712_175			
12	AB_202509251123_206			
13	AB_202509291932_545			
14	AB_202510031024_556			
15	AB_202510141758_275			
16	AB_202510291940_368			
17	AB_202511211633_810			
18	AB_202511261716_839			

三、变更详情

3.1 CartCustomGrayForTmallSwitch.java 变更

变更方式：
- 将已推全实验的 AB Key 置空（`""`）
- 注释已废弃的开关
- 新增替代开关（如 `TMALL_NGK_SWITCH`）

主要变更点：

行号	变更内容
L199	注释 `TMALL_GET_SHARE_AB_COMPONENT_ID`
L285	注释 `TMALL_FIND_PROMOTION_V2_AB_COMPONENT_ID`
L295	注释 `TMALL_FIND_PROMOTION_NEW_V2_AB_COMPONENT_ID`
L468	注释 `buy120AbKey`
L641	注释 `TMALL_ENABLE_SHOP_RED_PACKET_URGE_EXP`、`TMALL_SHOP_RED_PACKET_URGE_EXP_KEY`
L718	`TMALL_CROSS_TIME_PLATFORM_COUPON_SHOP_URGE_SUPPORT_MODEL` AB Key 置空
L786	注释 `TMALL_NGK_AB_KEY`
L790	**新增** `TMALL_NGK_SWITCH = true`
L817	注释 `TMALL_CONSUME_COUPON_PRICE_SUPPORT_AB`、`TMALL_CONSUME_COUPON_PRICE_AB_KEY`
L836	`TMALL_CATEGORY_COUPON_APPLY_SUPPORT` AB Key 置空
L854	`TMALL_CLOTH_NN_APPLY_SUPPORT` AB Key 置空
L964	`TMALL_UPDATE_FILTER_AB_KEY` 置空
L1045-1055	`TMALL_NGK_CHECK_SUPPORT_MODEL` 等 3 处 AB Key 置空
L1063	注释 `TMALL_FILTER_UPDATED_SKIP_AB_SWITCHER`
L1078	`TMALL_XSD_CHECK_SUPPORT_MODEL` AB Key 置空
L1101	`TMALL_VIPPURCHASECARD_ADDON_GRAY` AB Key 置空
L1106	`TMALL_MERGE_TRACK_GRAY` AB Key 置空
L1160	`vipPurchaseCardTrackGrayForTmall` AB Key 置空
L1166	`multiTrackUseNgkForTmall` AB Key 置空
L1181	`ALIPAY_TMALL_YJCD_SUPPORT` AB Key 置空
L1212	`TMALL_ITEM_CARD_VOUCHER_OPT_MODEL` AB Key 置空
L1243	`TMALL_88VIP_OPEN_CARD_GRAY` AB Key 置空
L1264	`TMALL_FOOT_BAR_UNIFY_PRIORITY_GRAY` AB Key 置空

3.2 CartCustomGrayForTmallUtil.java 变更

变更方式： 注释 AB 实验判断逻辑，直接返回 true 或使用开关

方法	变更内容
`getShare()`	注释 AB 判断，返回 `true`
`isFindPromotionV2NewUI()`	注释 AB 判断，返回 `true`

四、测试建议

4.1 功能验证点

功能模块	验证点	预期结果
购物车分享	分享按钮展示	正常展示（原实验组效果）
找优惠迭代v2	新UI样式展示	展示新UI



4.2 回归测试范围



4.3 降级预案

如发现问题，可通过 Switch 开关快速降级：

功能	降级开关
内购卡切卡	`TMALL_NGK_SWITCH` 设为 `false`
其他 DefaultBizSupportModel 功能	对应 Model 的 `open` 设为 `false`

五、风险评估

风险项	风险等级	说明
功能回退	低	已推全实验，用户已习惯新功能
代码兼容性	低	仅注释 AB 判断，保留原有灰度逻辑
性能影响	低	减少 AB 查询，可能略有提升

六、未圈化实验

以下实验**未在本次变更范围**：

AB 实验 Key	状态	说明



七、DefaultBizSupportModel 开关变更清单

> ⚠️ **重要提醒**：以下开关类型为 `DefaultBizSupportModel`，其值可能已被 Switch 平台持久化。
>
> **发布前需确认**：查询 Switch 平台持久化值，如持久化值中 abKey 不为空，需要执行 Switch 变更推送，将 abKey 置空。

序号	开关名称	业务标识	原 AB Key	说明





就这样，七句话，一年积压的AB实验下线工作，搞定！

03 核心思路：把复杂问题拆解成简单步骤

先把工具准备好，不要急着动手

很多人一看到任务就急着开始写代码，结果写到一半发现方向错了，还得推倒重来。我的做法是先花时间把工具准备好：

- **ab-experiment-query**：查询单个实验状态的工具
- **scan-ab-experiments**：批量扫描和生成报告的工具
- **upgrade-snapshot**：版本升级的工具

有了这些工具，后面的工作就像搭积木一样简单。

工具化关键步骤，让机器干活

人工操作最大的问题就是容易出错，而且效率低。我把关键步骤都工具化了：

- **实验检索**：用grep从代码中提取AB实验key
- **状态查询**：通过IDEAs平台API批量查询实验状态
- **报告生成**：自动生成markdown格式的报告，表格清晰易读
- **代码固化**：批量修改代码，添加注释说明

这样，我只需要告诉CodingAgent"做什么"，而不是"怎么做"。

创建skill，让工具协同工作

单个工具虽然好用，但如果能组合起来就更强大了。我创建了三个skill，每个skill都是一个完整的工作流：

```
# ab-experiment-query skill
name: ab-experiment-query
description: 根据 AB 实验 key 查询实验状态
workflow:
- 获取用户输入的实验key
- 调用IDEAs API查询状态
- 解析并展示结果
# scan-ab-experiments skill
name: scan-ab-experiments
description: 扫描代码中的 AB 实验，查询状态并生成报告
workflow:
- 扫描代码提取实验key
- 批量查询实验状态
- 筛选已推全实验
- 生成markdown报告
# upgrade-snapshot skill
name: upgrade-snapshot
description: 升级二方库版本为 SNAPSHOT
workflow:
- 读取pom.xml获取当前版本
- 版本号+1
- 提取分支标识
- 生成新版本号
- 更新pom.xml
```

这样，CodingAgent就知道什么时候该用什么工具，整个流程就像流水线一样顺畅。

生成报告，把控过程

很多人觉得写报告是浪费时间，但我觉得报告是控制过程的最好方式。每完成一个步骤，就生成一份报告：

- **实验状态报告**：告诉我哪些实验已推全，哪些还在运行
- **代码固化报告**：告诉我修改了哪些文件，改了哪些实验

- **提测报告：**汇总所有操作，方便测试

有了这些报告，我就能随时掌握进度，及时发现问题。

明确的编码指令，批量完成改造

CodingAgent最强大的地方就是能理解自然语言指令。我给它的指令非常明确：

"把报告文件中已推全的实验逻辑，固化到代码中。固化方式：将AB实验key设置为空值，添加注释说明原值。"

CodingAgent就会按照这个指令，自动完成所有操作：

```
// 修改前
public static final String AB_202407291048_1537 =
"AB_202407291048_1537";

// 修改后
// [AB已推全-2026.02.02] 原值: AB_202407291048_1537
public static final String AB_202407291048_1537 ="";
```

人工review，确保质量

虽然CodingAgent很厉害，但我觉得还是需要人工介入。我会仔细review报告，检查：

- 实验是否真的可以下线
- 代码修改是否正确
- 是否有遗漏的实验
- 注释是否完整

发现问题就及时修正，确保万无一失。

生成提测报告，准备上线

最后一步是生成提测报告，这份报告包括：

- 修改的文件列表
- 固化的实验清单
- 每个实验的改造摘要

- 风险评估
- 测试建议

有了这份报告，测试同学就知道该怎么测试了。

04 实战经验：踩过的坑和学到的教训

坑1：API调用失败

刚开始查询实验状态的时候，我用的是钉钉个人助理接口，结果返回的是success，但实际结果要通过钉钉消息异步返回。



后来我改用了IDEAs平台的API，把API文档生成md文档交给CodingAgent学习适配，问题就解决了。



所以，遇到问题不要着急，不要轻易放弃。不要预设困难，找到文档让AI学习，他的能力可能比你想象中要强！

坑2：JSON解析错误

批量查询实验状态的时候，有些API返回的JSON包含控制字符，导致解析失败。

教训：不要相信API返回的数据永远是干净的，但AI可以处理这种问题，只需要你给出明确的引导。

坑3：版本号冲突

升级版本号的时候，如果多个开发者同时开发，可能会出现版本号冲突。我让CodingAgent从分支名提取功能标识，加到版本号里，这样就避免了冲突。

教训：版本号管理要考虑团队协作场景。

坑4：代码固化策略选择

一开始我想直接删除AB判断逻辑，但这样改动太大，风险太高。后来选择了将key设置为空值，这样改动最小，而且保留了原有变量定义，后续可以逐步清理。

教训：重构要循序渐进，不要一步到位。

05 总结：AI不是替代，而是增强

有人说AI会取代程序员，我觉得不会。AI更像是一个超级助繁琐的工作，让我们把精力花在更有价值的事情上。手，它能帮我们完成那些重复、就像这次AB实验下线，如果手动做，可能需要一周时间。但有了CodingAgent的帮助，我只需要花一天时间思考和review，剩下的工作都交给CodingAgent完成。当然，前提是你得会"指挥"CodingAgent。你得知道什么时候该让它做什么，怎么给它明确的指令，怎么review它的结果。这其实也是一种能力。所以，不要害怕AI，要学会利用AI。毕竟，我们的目标是解决问题，而不是展示自己有多能干。

> 写在最后
> 七句话只是调侃，其实黎明前有很多人在黑夜里默默流汗。

06 团队介绍

本文作者依凡，来自淘天集团—天猫APP技术团队。作为淘天集团的核心技术引擎，我们负责天猫APP的全栈开发与体验升级，支撑导购、搜推、商详、购物车、下单等核心交易链路。我们以极致的端侧性能与创新的Agent架构，打造「AI导购」「AI助手」等面向C端的智能应用，并探索端外场景合作，为千万级用户构建沉浸式、个性化的购物体验，持续引领电商APP的技术演进。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术
服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 2463

留言

写留言

还没有留言，来写首评